

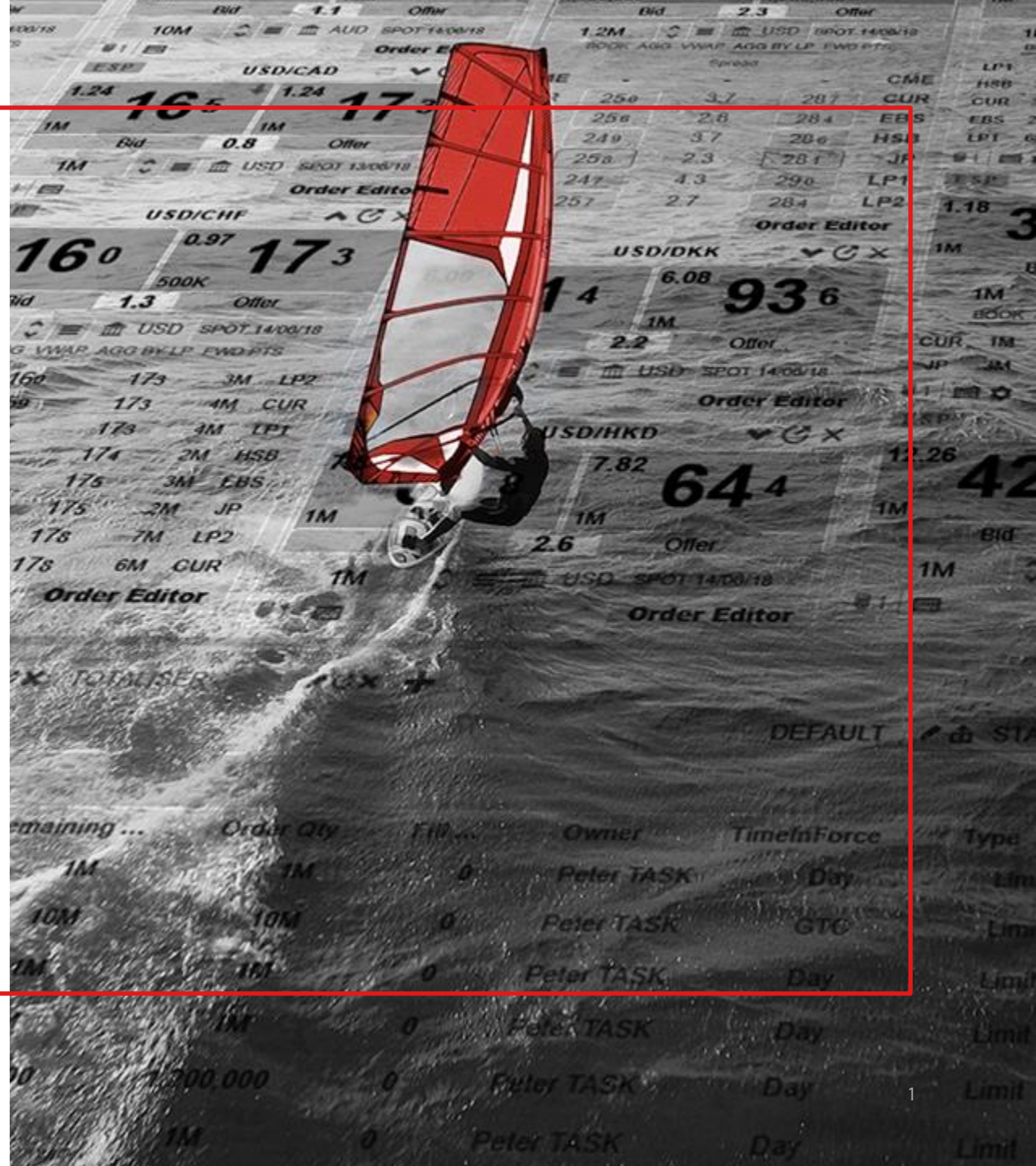


Using Generative AI to Predict Data
to Improve Latency at the Client

Stagiaire de recherche
Supervisor : Mathieu

AI Group

Shubhankar



Agenda

- 1 Introduction
- 2 Problem Statement
- 3 Related Work
- 4 Implementation
- 5 Future Work
- 6 Q & A

Introduction

Aim

- Explore generative AI for byte-level prediction and data compression at the client side.
- Reduce frequent server communications by predicting data, thereby lowering latency.

Introduction

Problem Statement

- **Hypotheses** : Investigate transformer-based generative models to enhance prediction accuracy and efficiency.
- Expectation: A runtime-adaptive dictionary acting as a specialized cache for server-to-client transmissions.

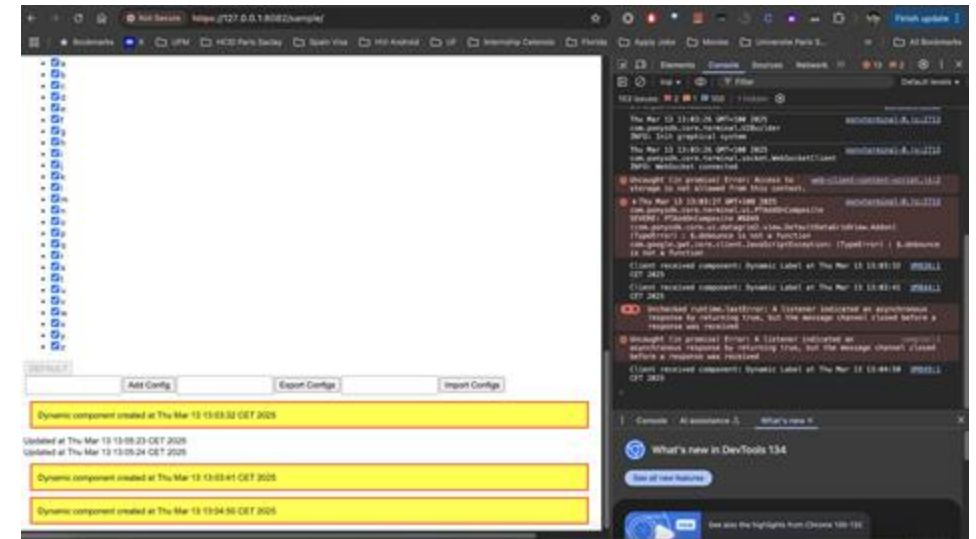
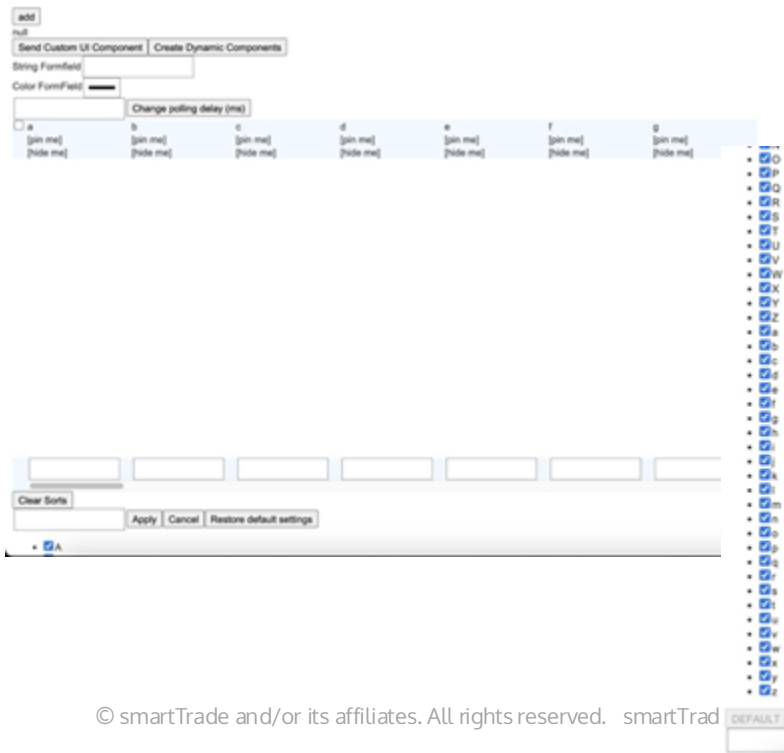


- Step 1: Static Dictionary Approach: Leverage known data values (as in PonySDK) by representing them with concise codes.
- Step 2: Dynamic Dictionary Model : Develop a system to detect and map known paths in a directed graph using



Introduction

- **Current Limitations:**
 - Traditional optimizations (network tweaks, caching) fall short when data is dynamic.
 - WebSocket-based communications (via PonySDK) currently encode structured data (primitive types, strings, arrays) sequentially, causing inefficiencies.



Problem Statement

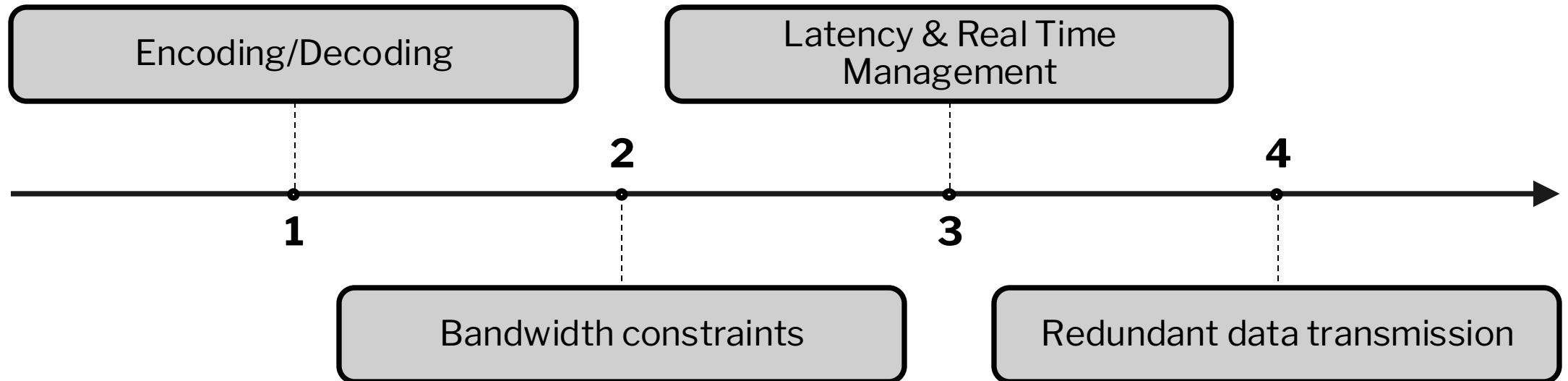


- Able to encode something we want to predict
- Do not encode something we can not predict

Problem Statement

Research Focus?

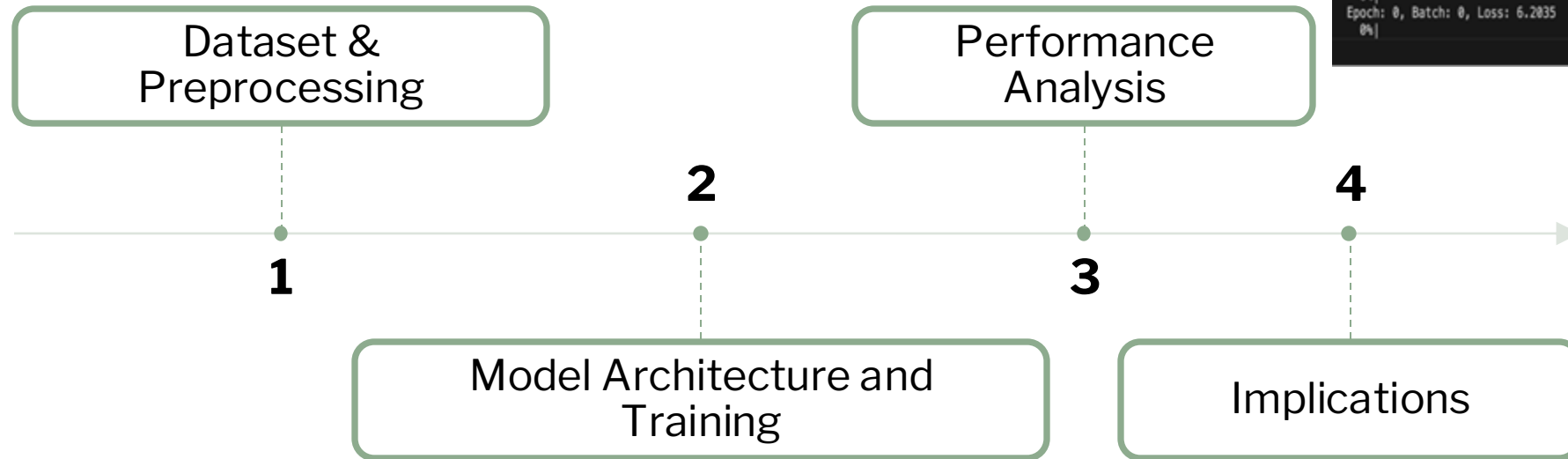
- Leverage transformer-based generative models
- Develop a dynamic dictionary approach to identify recurring data paths.



Implementation



```
UnboundLocalError: cannot access local variable 'eval_loss' where it is not associated with a value
Epoch 1/3: 0% | 0/3750 [00:16<?, ?it/s, loss=5.75]
(base) shubhankar@Sahadevas-MacBook-Air bgpt_project % export PYTORCH_ENABLE_MPS_FALLBACK=1
(base) shubhankar@Sahadevas-MacBook-Air bgpt_project % python train-gen.py
Using device: mps
Loading 1000 files
Loading 200 files
Epoch 1/2: 100% | 32/32 [00:46<00:00, 1.44s/it, loss=2.77]
Epoch: 1, Train Loss: 3.4574, Eval Loss: 2.9628
Epoch 2/2: 100% | 32/32 [00:44<00:00, 1.40s/it, loss=2.86]
Epoch: 2, Train Loss: 2.7184, Eval Loss: 2.6664
(base) shubhankar@Sahadevas-MacBook-Air bgpt_project %
```



```
(base) shubhankar@Sahadevas-MacBook-Air bgpt_project % python train-gen.py
Using device: cpu
Epoch: 0, Batch: 0, Loss: 6.2035
0% | 0/15000 [00:00<?, ?it/s]
2/15000 [03:04<390:09:11, 93.65s/it]
```

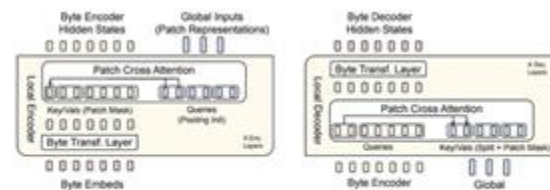


Figure 5 The local encoder uses a cross-attention block with patch representations as queries, and byte representations as keys/values to encode byte representations into patch representations. The local decoder uses a similar block but with the roles reversed i.e. byte representations are now the queries and patch representations are the keys/values. Here we use Cross-Attn & $\kappa = 2$.

Prompt: "The company announced"

Generated: "The company announceda beang anine s s u in n in Sain che aint on ont t s r a s tore bor in ar a ing as f lan a mi int al"

Classification & Compression

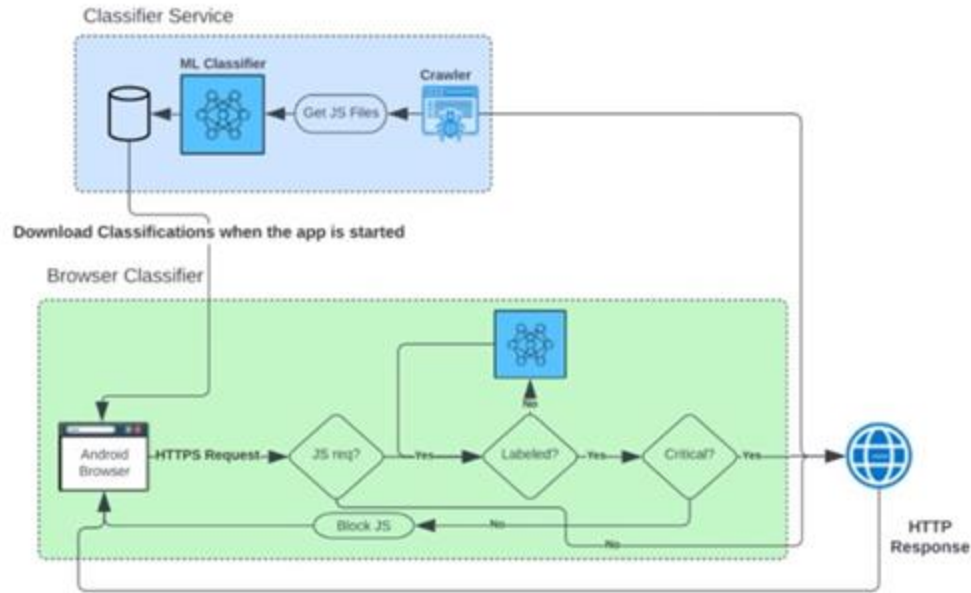


Figure 2: Browser Architecture

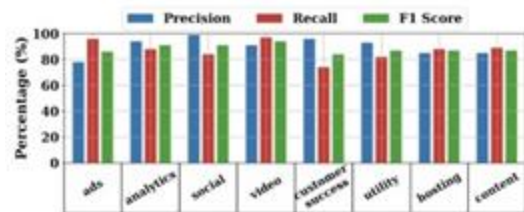


Figure 1: Summary of the 4-layer Neural Network from the SlimWeb Paper

Algorithm 1 Classifying the JavaScript

```

1: Input: string script
2: Output: category
3: category, predictionProb ← MLClass(script)
4: if predictionProb > 0.8 and category ∈ {ads, marketing,
   socials, analytics} then
5:   return category
6: else
7:   return unassigned_category
8: end if
    
```



Figure 1. A flow chart of our algorithm. Given a text dataset, the LZW algorithm is implemented to generate a dictionary either at word-level or character-level. Then, the dictionary is updated by selecting a subset of atoms with high discriminative to better accomplish classification tasks. Labels are taken into consideration during the selection. With the updated dictionary, the text dataset is vectorized into a sparse vector representation. Finally, simple classifiers such as SVMs or neural networks can be trained for classification tasks.

Byte Transformer

- Process raw bytes
 - without intermediate tokenization.
- Employ patching mechanisms
 - to break long sequences into manageable units.
- Hierarchical designs:
 - local (intra-patch) and global (inter-patch) modules.
- Dual-decoder strategies
 - (e.g., patch-level decoder and byte-level decoder).
- Ex: ByteGPT, MegaByte, MambaByte, Byte Latent Transformer.

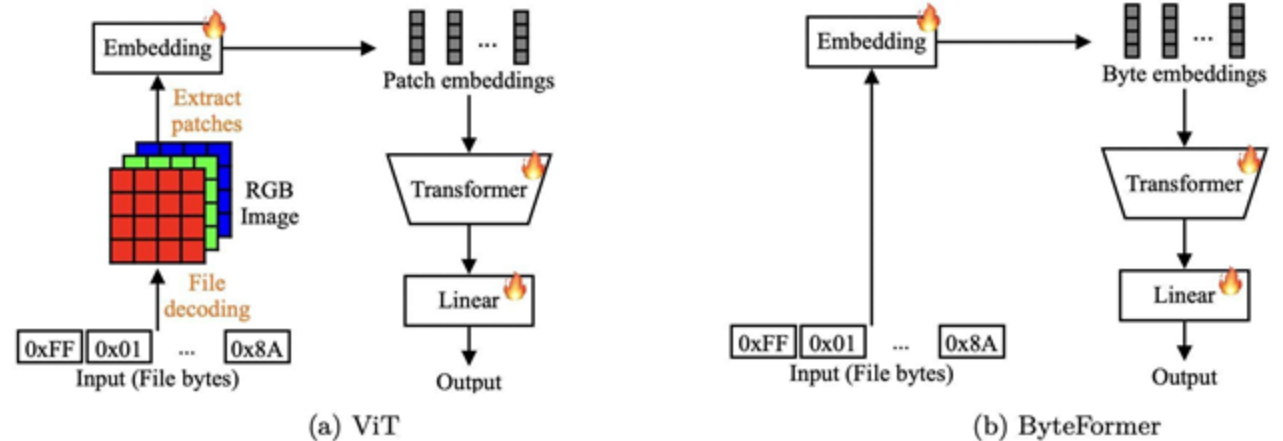
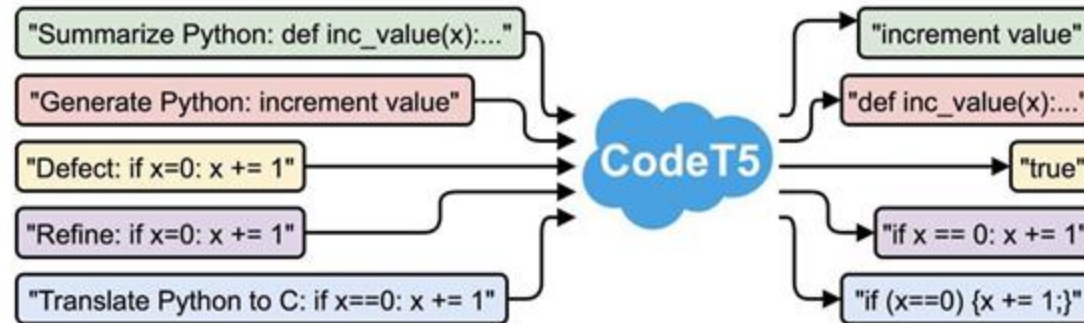


Figure 1: **ByteFormer vs. ViT.** (a) A standard vision Transformer (ViT) decodes file bytes into an RGB image. Subsequently, the image is split into patches and patch embeddings are extracted and fed to Transformer to obtain contextualized patch embeddings, which are then classified using a linear classifier. (b) ByteFormer directly operations on file bytes.

why we will not use it

Byte Transformer : Code T5



```
1 // convert from one currency to another
2 public static String convertCurrency(String fromISO, String toISO, String fromCurrency, String toCurrency) {
3     if(fromISO == toISO) // no need to convert
4         return null;
5     else //return the value to the currency
6         return UTIL_Currency.getInstance().convertFromISO(fromISO, toISO, fromCurrency);
7 }
8
9 // returns whether the contact's mailing address is empty
10 private boolean isContactAddressEmpty(Contact con1)
11 {
12     boolean isEmpty =
13         con1.MailingStreet == null &&
14         con1.MailingCity == null &&
15         con1.MailingState == null &&
16         con1.MailingPostalCode == null &&
17         // note that we decided to ignore country so that a default value won't create unnecessary address objects
18         (con1.MailingCountry == null || ADDR_Addresses.TOTM.IsStateCountryPicklistsEnabled) &&
19         // only test StateCode if picklists enabled.
20         (ADDR_Addresses.TOTM.IsStateCountryPicklistsEnabled && con1.StateCode == null);
21     return isEmpty;
22 }
```

Code Summarization

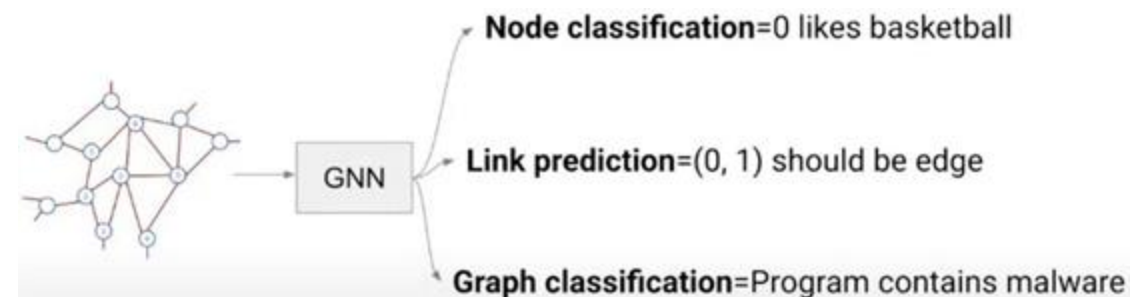
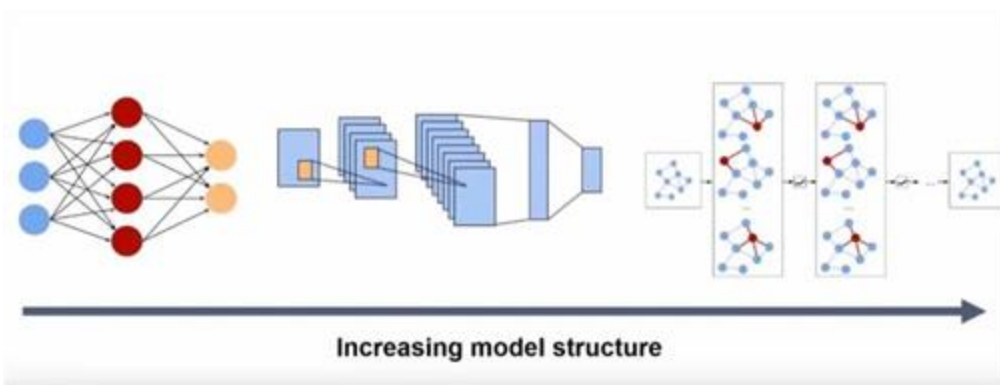
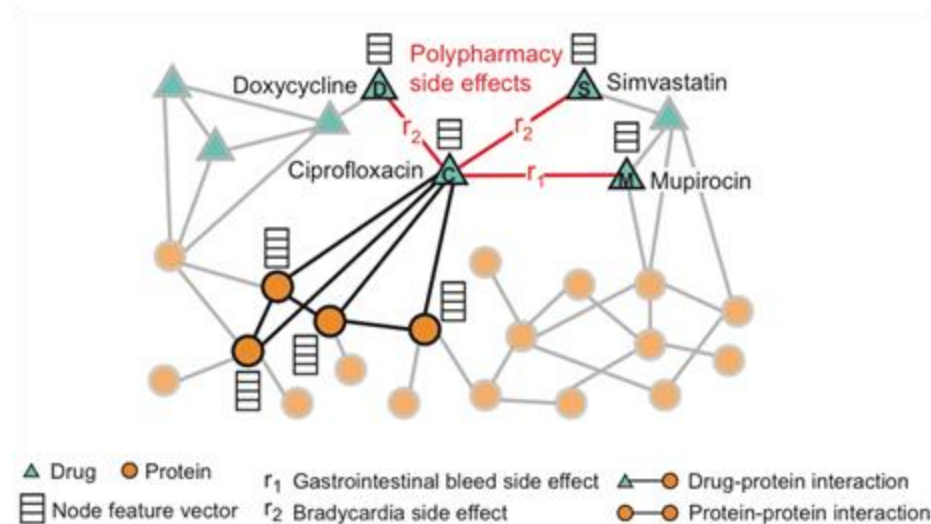
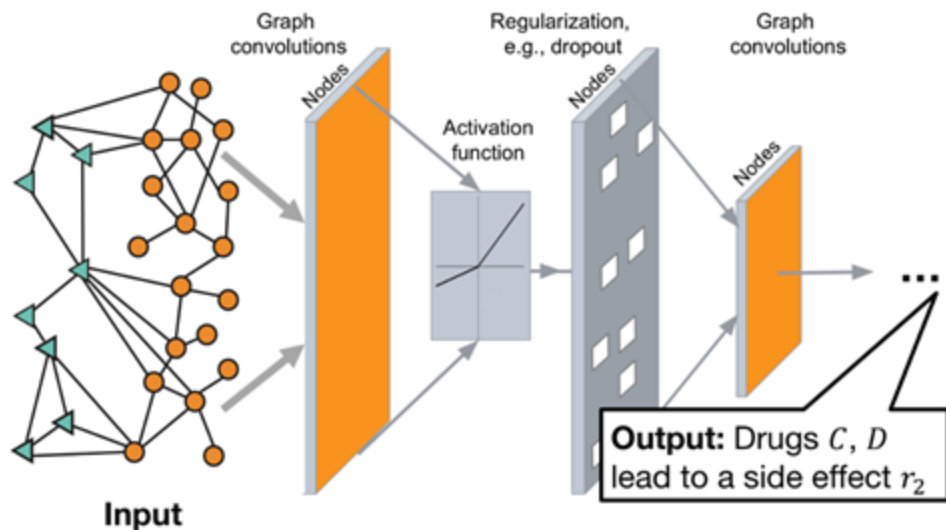
```
1 // convert from one currency to another
2 public static String convertCurrency(String fromISO, String toISO, String fromCurrency, String toCurrency) {
3     if(fromISO == toISO) // no need to convert
4         return null;
5     else //return the value to the currency
6         return UTIL_Currency.getInstance().convertFromISO(fromISO, toISO, fromCurrency);
7 }
8
9 // returns whether the contact's mailing address is empty
10 private boolean isContactAddressEmpty(Contact con1)
11 {
12     boolean isEmpty =
13         con1.MailingStreet == null &&
14         con1.MailingCity == null &&
15         con1.MailingState == null &&
16         con1.MailingPostalCode == null &&
17         // note that we decided to ignore country so that a default value won't create unnecessary address objects
18         (con1.MailingCountry == null || ADDR_Addresses.TOTM.IsStateCountryPicklistsEnabled) &&
19         // only test StateCode if picklists enabled.
20         (ADDR_Addresses.TOTM.IsStateCountryPicklistsEnabled && con1.StateCode == null);
21     return isEmpty;
22 }
```

Text-to-Code Generation

```
1 // convert from one currency to another
2 public static String convertCurrency(String fromISO, String toISO, String fromCurrency, String toCurrency) {
3     if(fromISO == toISO) // no need to convert
4         return null;
5     else //return the value to the currency
6         return UTIL_Currency.getInstance().convertFromISO(fromISO, toISO, fromCurrency);
7 }
8
9 // returns whether the contact's mailing address is empty
10 private boolean isContactAddressEmpty(Contact con1)
11 {
12     boolean isEmpty =
13         con1.MailingStreet == null &&
14         con1.MailingCity == null &&
15         con1.MailingState == null &&
16         con1.MailingPostalCode == null &&
17         // note that we decided to ignore country so that a default value won't create unnecessary address objects
18         (con1.MailingCountry == null || ADDR_Addresses.TOTM.IsStateCountryPicklistsEnabled) &&
19         // only test StateCode if picklists enabled.
20         (ADDR_Addresses.TOTM.IsStateCountryPicklistsEnabled && con1.StateCode == null);
21     return isEmpty;
22 }
```

Code Autocompletion

Graphs?



Gates?

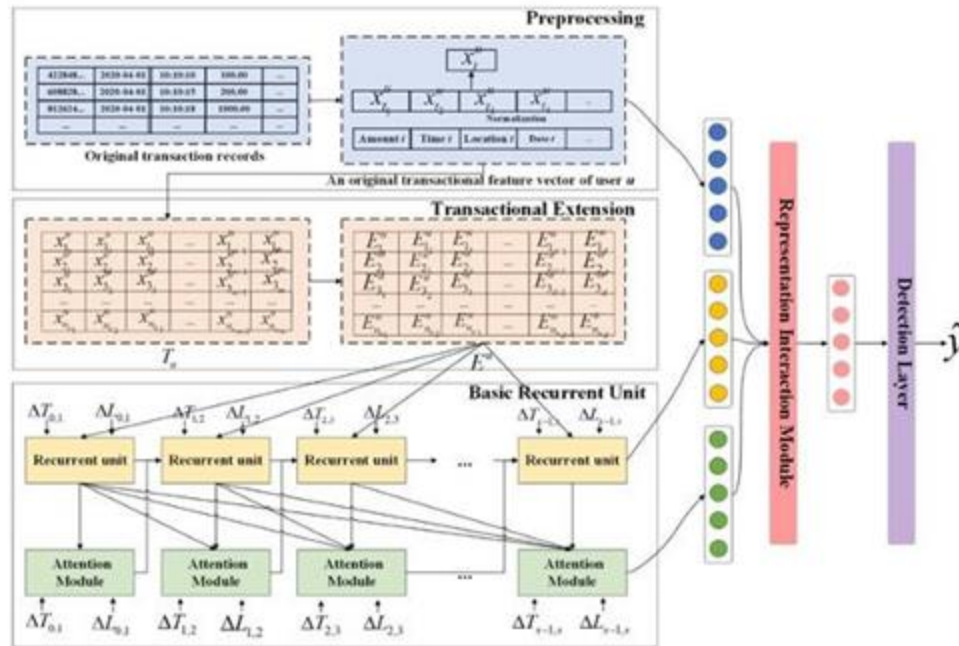


Fig. 2. Framework of STGN for CCFD.

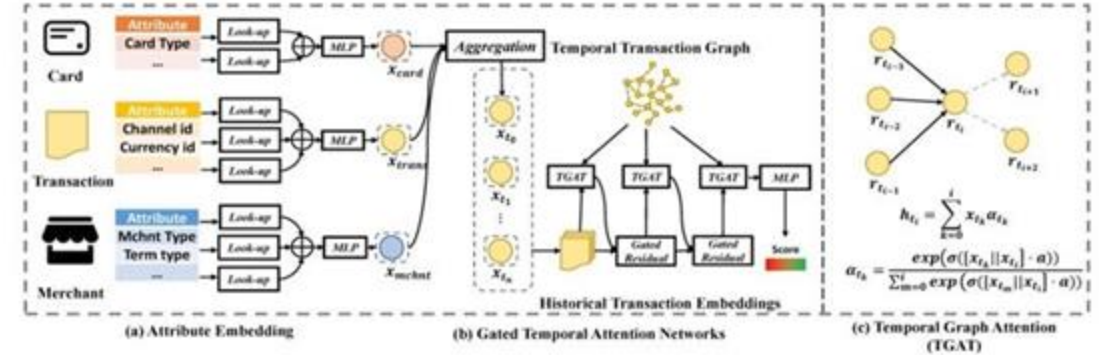


Figure 2: The illustration of the proposed model architecture and temporal graph attention mechanism.

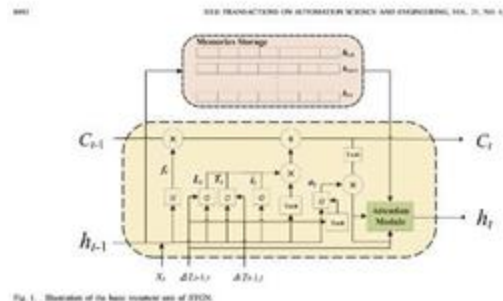
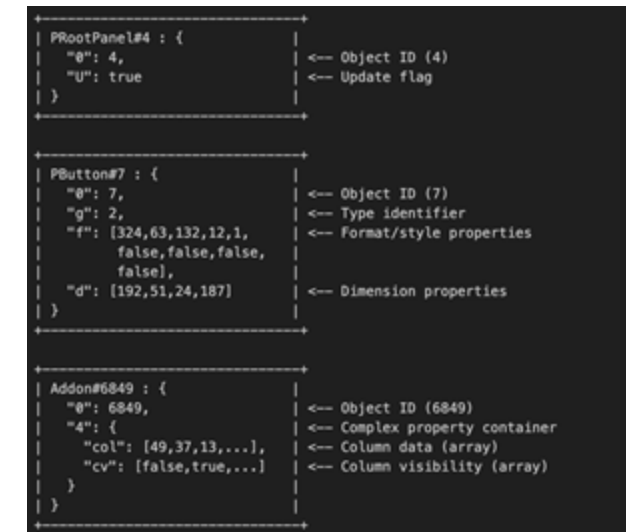
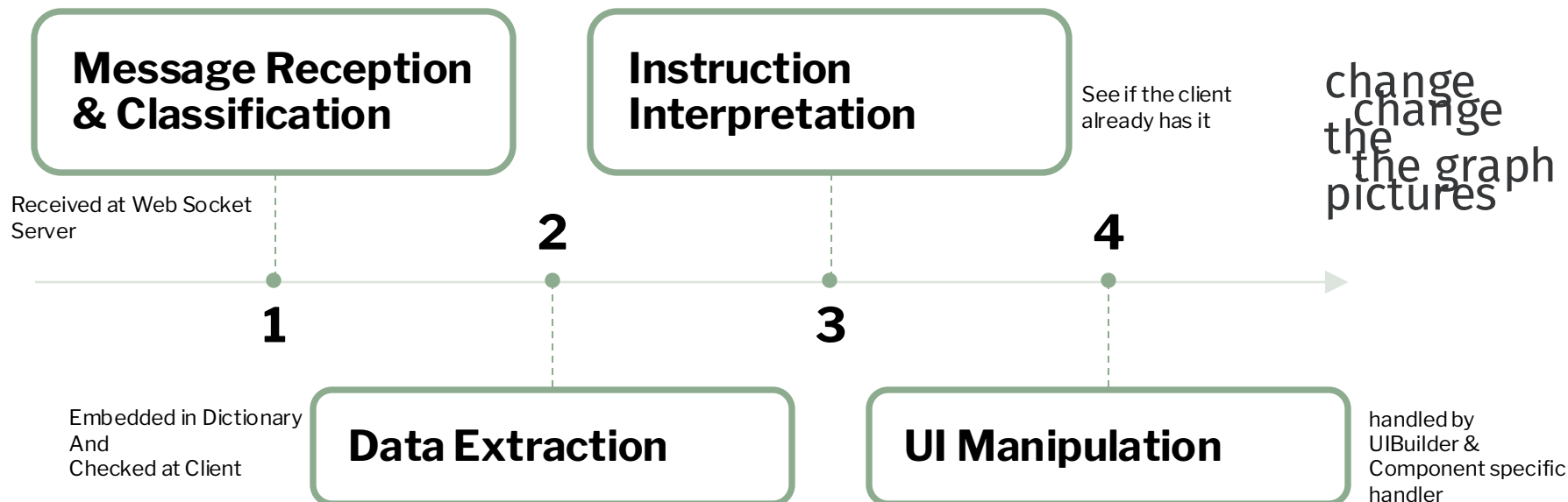
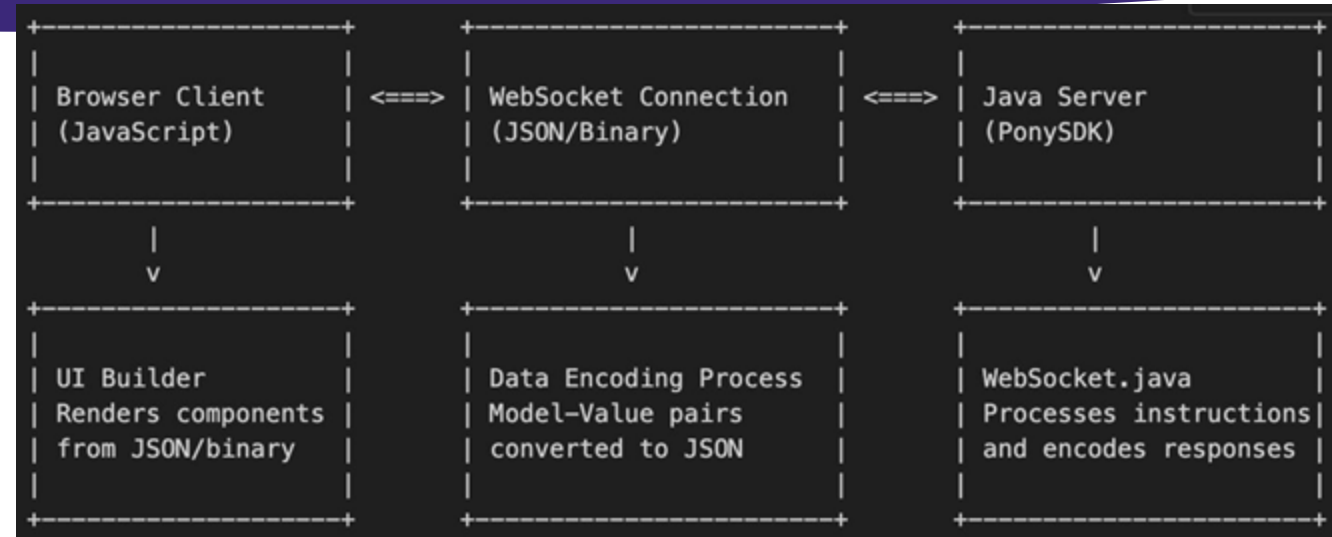


Fig. 3. Illustration of the basic recurrent unit of STGN.

Mise à jour du statut : 1 mois

Document Object Model - it's the programming interface for web documents that represents the page structure as a tree of objects



Future Plan

what to do with dynamic dictionary
updating dictionary
pushing in server and client side
all the important things, also explained
and with schema

Step 1 : static (ongoing)
step 2: dynamic

References

- ByteGPT: Beyond Language Models for Byte-Level Digital World Simulation. Zheng, Yao et al. 2024. arXiv preprint arXiv:2402.19155.
- MegaByte: Predicting Million-byte Sequences with Multiscale Transformers. Yu, Lili et al. 2023. arXiv preprint arXiv:2305.07185.
- MambaByte: Token-Free State Space Models for Byte-Level Language Modeling. Dao, Tri et al. 2024. arXiv preprint arXiv:2401.13660.
- ByT5: Towards a token-free future with pre-trained byte-to-byte models. Xue, Linting et al. 2021. Transactions of the Association for Computational Linguistics.
- CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation. Wang, Yue et al. 2021. EMNLP 2021.
- Byte Latent Transformer: Patches Scale Better than Tokens. Meta AI Research. 2024. Meta AI Research Publication.
- DocScript: Document-level Script Event Prediction.. 2024. LREC 2024.
- Annotating FrameNet via Structure-Conditioned Language Generation. 2024. arXiv preprint arXiv:2406.04834.
- How Powerful are Graph Neural Networks? Xu, Keyulu et al. 2018. arXiv preprint arXiv:1810.00826.
- CodeBERT: A Pre-Trained Model for Programming and Natural Languages. Feng, Zhangyin et al. 2020. EMNLP 2020.
- CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation. Lu, Shuai et al. 2021. arXiv preprint arXiv:2102.04664.
- Accelerating Mobile Web Performance through JavaScript Classification on the Browser. Silwal, Sashank. 2023. NYUAD Capstone Project 2 Reports, Spring 2023 .
- Lightweight Text Classification Using LZW Compression. Wan, Li et al. 2024.

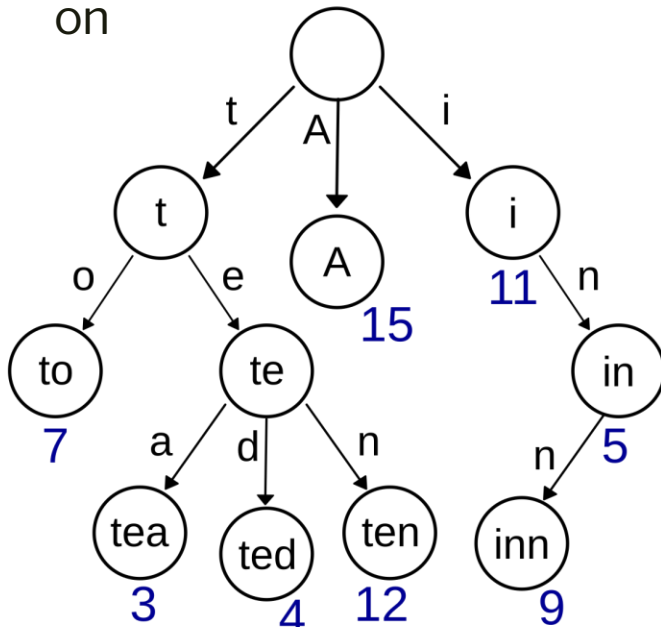
Trie

Buffer: keep a `List<String>` buffer of the last 0–2 instructions

-

Why?

- Faster lookup
- Avoid sending individual elements along for prediction that it has not been trained on



On new event:

1. Add to buffer → check `buffer.size()`

2. If `size==1` → “Is this a valid first step?”
(`isPrefixOfKnownTriplet`)

3. If `size==2` → “Is this a valid 2-step prefix?”

- **Yes** → send those two to /generate

- **No** → flush first, reprocess second

Process

Followup

Task 1: Improve Transformer Model (TD)ULL – Ultra Low Latency

Task 2: Inject TD into Sample Pony SDK

Task 3: In Pony use Task 2 + Dictionary

Task 4: In SMS UI deal with static dictionary

Task 5: In SMS UI deal with Task 2 + Task 3

What we discussed last few times?

- A small demo with Postman
- Had file with static data
- Check how the generated data compares with the predicted data
- Similarity function with generated data and static data
- Have a way to directly connect the Java System with FastAPI and transformer

Update

Trie

- Works with local smaller sets of things
- We tried using logs directly to capture P..., the structure

Dictionary

- Works with static components
- Able to store snapshot

Next Steps:

Changes

SV1->
SV2->
SV3

SV4->
SV5->
SV6

S, V -> Transformer T5

S, V = Server Client Model, Value

Validate if $SV == P$ (Predicted data)

We have dataset, similarity function,

Dictionary

```
[ TYPE_CREATE(42),  
  WIDGET_TYPE("PButton"),  
  TEXT("Click me"), END ]
```



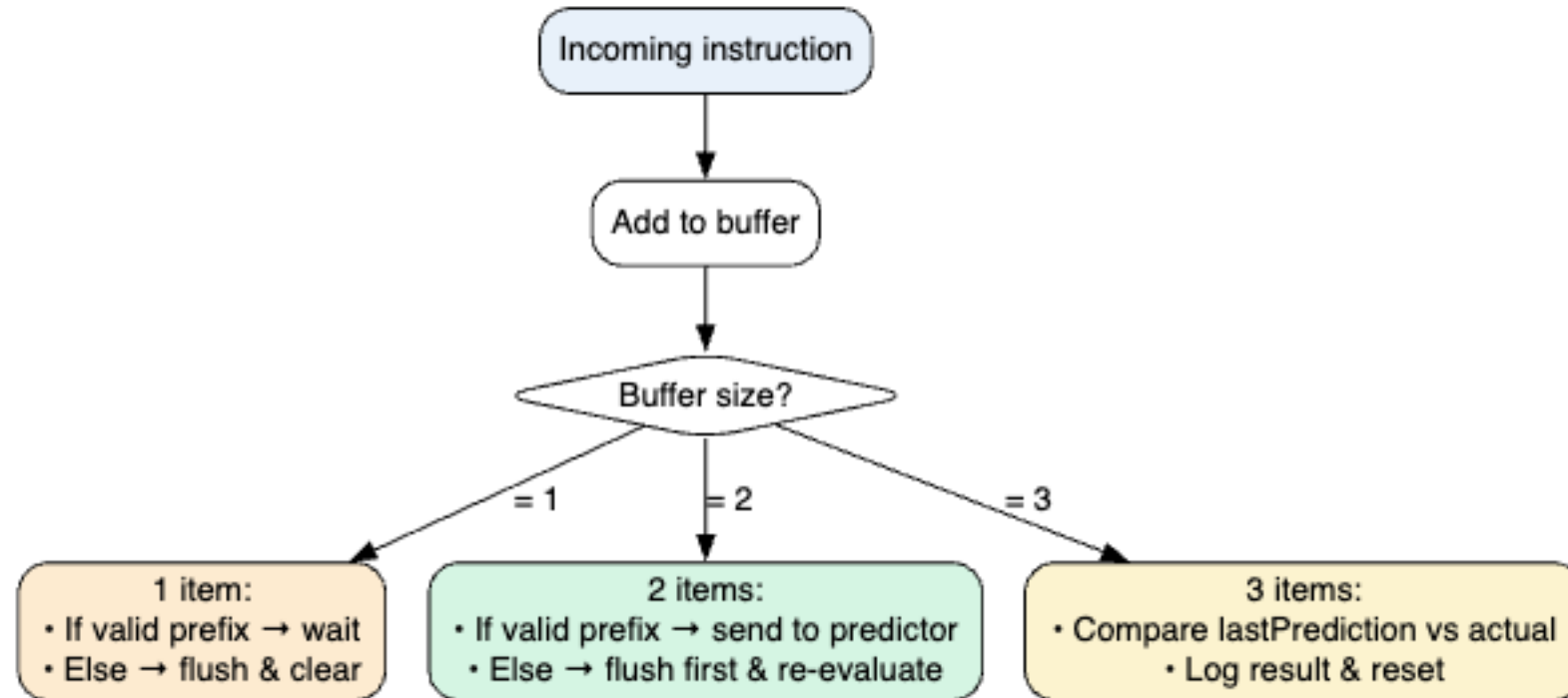
```
"TYPE_CREATE -> WIDGET_TYPE -> TEXT -> END"
```

```
List<ModelValuePair> pattern = dictionary.getPattern(id);
```

```
String seq = pattern.stream()  
    .map(p -> p.getModel().name())  
    .collect(Collectors.joining(" -> "));
```

```
PRED.info("Pattern #{}: {}", id, seq);
```

Trie



Trie

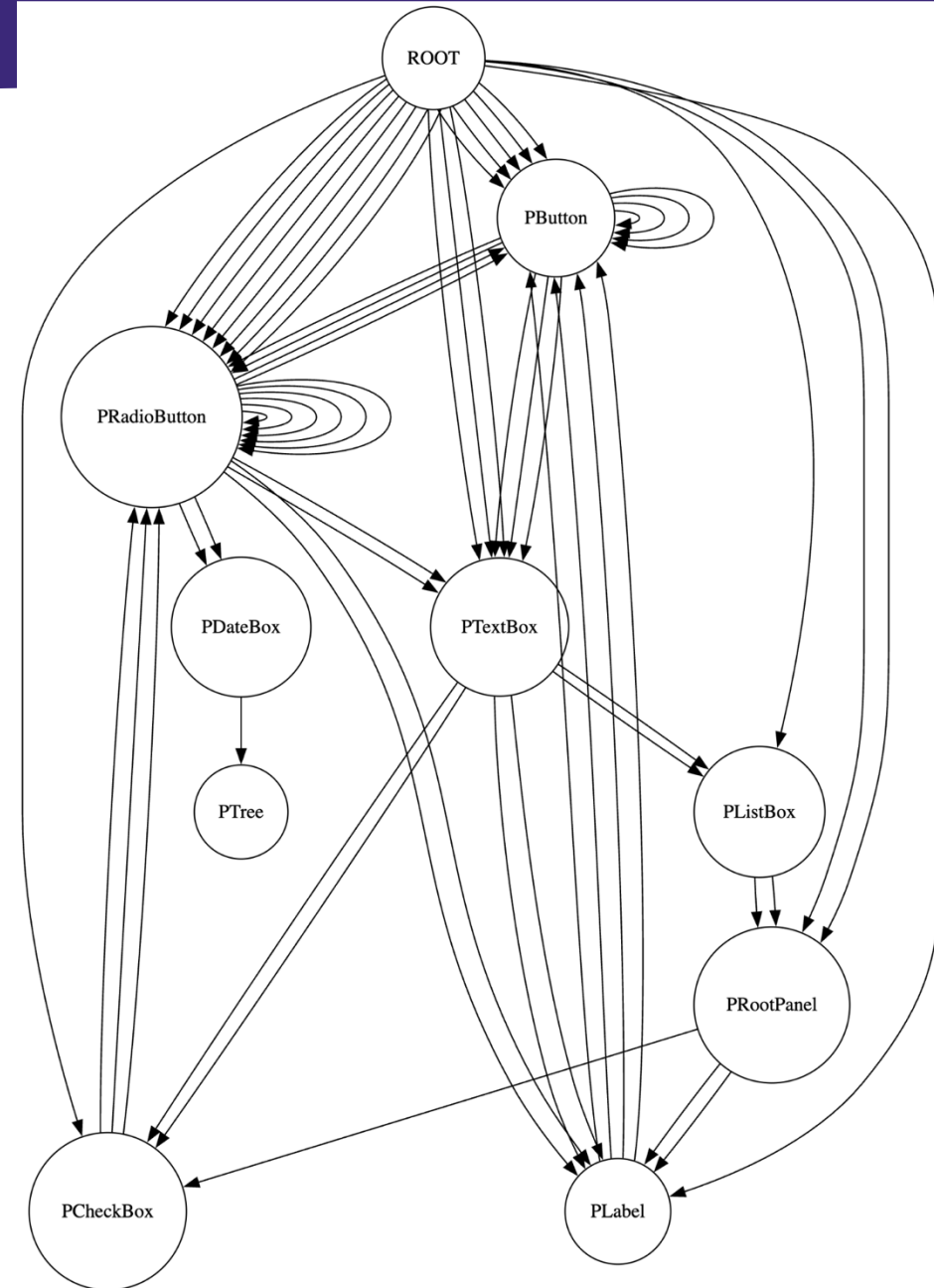


```

digraph Trie {
  node [shape=circle];
  ROOT [label="ROOT"];
  ROOT -> "PButton#11" -> "PTextBox#12" ->
    "PLabel#44";
  ROOT -> "PButton#6" -> "PButton#8" ->
    "PRadioButton#10";
  ROOT -> "PButton#7" -> "PButton#6" ->
    "PButton#8";
  ROOT -> "PButton#8" -> "PButton#7" ->
    "PButton#6";
  .....

```

.....



Log

```
PButton#8, text=Static Component, html=null :
{"O":8,"g":2,"f":[469,60,92,8,1,false,false,false,false],"d":[377,52,23,119]}
10:52:19.936 INFO qtp1037475674-28 [PredictionLogger]
Model S2C TYPE_CREATE 54
10:52:19.936 INFO qtp1037475674-28 [PredictionLogger]
Model S2C WIDGET_TYPE 12
10:52:19.936 INFO qtp1037475674-28 [PredictionLogger]
Model S2C ADD_STYLE_NAME static-component
10:52:19.936 INFO qtp1037475674-28 [PredictionLogger]
Model S2C TEXT Static component created
10:52:19.936 INFO qtp1037475674-28 [PredictionLogger]
Model S2C END null
10:52:19.936 INFO qtp1037475674-28 [PredictionLogger]
Model S2C TYPE_ADD 54
10:52:19.936 INFO qtp1037475674-28 [PredictionLogger]
Model S2C PARENT_OBJECT_ID 4
10:52:19.936 INFO qtp1037475674-28 [PredictionLogger]
Model S2C END null
10:52:19.936 INFO qtp1037475674-28 [PredictionLogger]
Model S2C TYPE_UPDATE 8
10:52:19.936 INFO qtp1037475674-28 [PredictionLogger]
Model S2C END_OF_PROCESSING null
10:52:19.936 INFO qtp1037475674-28 [PredictionLogger]
Model S2C END null
DEBUG: processInstructions entered.
```

```
PButton#8, text=Static Component, html=null :
{"O":8,"g":2,"f":[469,60,92,8,1,false,false,false,false],"d":[377,52,23,119]}
10:52:20.116 INFO qtp1037475674-34 [PredictionLogger]
Model S2C TYPE_CREATE 55
10:52:20.116 INFO qtp1037475674-34 [PredictionLogger]
Model S2C WIDGET_TYPE 12
10:52:20.116 INFO qtp1037475674-34 [PredictionLogger]
Model S2C ADD_STYLE_NAME static-component
10:52:20.116 INFO qtp1037475674-34 [PredictionLogger]
Model S2C TEXT Static component created
10:52:20.116 INFO qtp1037475674-34 [PredictionLogger]
Model S2C END null
10:52:20.123 INFO qtp1037475674-34 [PredictionLogger]
Model S2C TYPE_UPDATE 8
10:52:20.123 INFO qtp1037475674-34 [PredictionLogger]
Model S2C END_OF_PROCESSING null
10:52:20.123 INFO qtp1037475674-34 [PredictionLogger]
Model S2C END null
```

Next Steps

- Fix the Trie for Triplet
- Train on SMS - Cockpit data
- Improve model fine tuning & Look into hyperparameters
- Feedback
- C correspond to value z, so you draw z, you draw a,b,c
- Third element received is d not c
- So revert the key z, display a,b,d -> made the mistake
- Log the file > calling for prediction > all the dictionary step working
- Performance optimization maybe enough
- Plug the prediction and train the prediction
- Prepare the correction, in case of prediction

Want to know what the server do?

Functionality of the server

What the client do

Client create event

Server receive notification of event and send notification

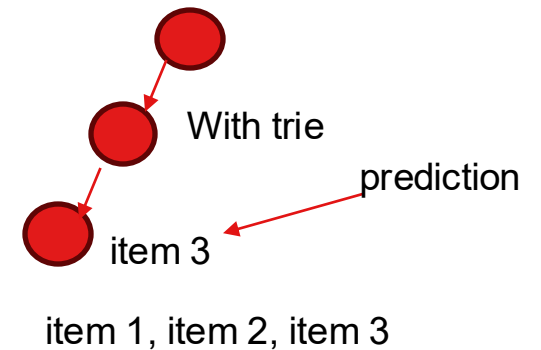
Now the flow of data, we know how to organize in some dictionary

It takes some key and give us some H

Dict [key] = h

H is some hash associated with some object

Trie Data structure for which you have item 1, item 2, item 3



This one give us the value, the key in fact

$D^{-1}[h] = \text{key if exist}$

Else

Ask for

$D: \{\text{Key}\} \dashrightarrow \{h\}$

Regarding some threshold , some frequency threshold

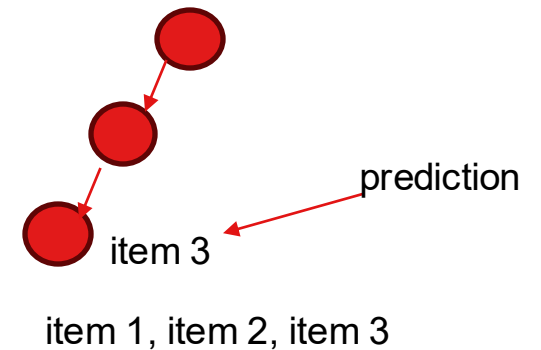
Client $\leftarrow$ ---- Socket -----> Server

[]

[]

$D^{-1}[h]$

$D, /\backslash \text{ Trie}$



Monitor network (analyse the quantity of data going through websocket from the server to the client)

Size of memory

Latency of treatment

Dynamicity

What we send to the keys > Some changing parameters

Some values we can change, We can keep track

A series of events

D: {Task 1, Task 2, Task 3}

Task breakdown of smaller tasks {}

Ok check etc. {[], [], []}

Feedback

Code Requirement

Need to

- Monitor network (analyse the quantity of data going through websocket from the server to the client)
- Size of memory
- Latency of treatment
- Procedure: How

Dynamicity

- What we send to the keys > Some changing parameters
- Some values we can change, We can keep track

A series of events

- D: {Task 1, Task 2, Task 3}
- Task breakdown of smaller tasks {}
- Ok check etc. {[], [], []}

Output

In WebSocketPerformanceTest.java (lines 49-50):

- `monitor = new NetworkMemoryLatencyMonitor();`
- `websocket.setListener(monitor); // <-- plug into websocket`

Now the monitor gets REAL callbacks from actual WebSocket operations:

- bytes sent - When WebSocket actually sends data, it calls `monitor.onOutgoingPonyFramesBytes(actualBytes)`
- frame counts - Each actual WebSocket frame triggers `monitor.onOutgoingWebSocketFrame()`
- acknowledgments - Jetty server's actual confirmation calls `monitor.onFrameWriteSuccess()`
- memory usage - `Runtime.getRuntime().totalMemory()` gives actual JVM heap

Problem solving focus?

Next steps

WebSocket Dictionary & Trie

Network Based Latency & Memory Tests

Testing with Data - Cockpit

Defining the academic setting



Q & A

